azar.
Laboratorio Monte Carlo

DEL PROYECTO AL APK / MANUAL PASO A PASO

React Native nativo + TypeScript. Backend Python y NumPy en Docker. Inicio de sesión con Google, experimentos privados en Firestore e histogramas de frecuencias relativas.

Entrega: fuentes Android completas, Gradle Wrapper, dependencias npm bloqueadas, backend, pruebas, reglas de Firestore y script de despliegue. La app no utiliza WebView ni Expo.

Antes de comenzar

Necesitas una cuenta de Google, un proyecto Google Cloud/Firebase propio y conexión a Internet. La configuración de Google, el despliegue y la firma se hacen con tus credenciales. No vienen claves privadas ni un backend desplegado en el ZIP.

Google Cloud puede requerir facturación. Cloud Run, Cloud Build, Artifact Registry y Firestore pueden generar cargos. El límite de instancias reduce capacidad, pero no fija un presupuesto monetario.

Ruta del manual

02 Arquitectura · 03 Herramientas · 04 Preparar fuentes · 05 Google Login · 06 Firestore · 07 Cloud Run · 08 Conectar Android · 09 APK firmado · 10 Prueba completa · 11 Variables · 12 Expresiones · 13 Protocolo · 14 Persistencia · 15 Límites · 16 Docker local · 17 Pruebas · 18 Errores · 19 Referencias · 20 Vectores · 21 Bloqueo y cupo diario · 22 Pruebas de ampliación.

Edición: 20 de octubre de 2026. Comandos Windows: PowerShell, excepto los identificados como Cloud Shell (Bash). Las rutas de ejemplo pueden adaptarse a tu equipo.

02 / Qué contiene el proyecto

```
MonteCarlo/
  mobile/                 React Native + TypeScript
    android/              Proyecto para Android Studio
    src/                  AST, API, distribuciones, histograma
    scripts/configure.mjs Configuración de tu backend y OAuth
    package-lock.json     Versiones npm resueltas
  backend/
    app/                  API, simulador, proceso y Firestore
    tests/                Pruebas automáticas
    Dockerfile            Python y NumPy fijados
    requirements.txt      Dependencias Python
  scripts/deploy.sh        Despliegue Cloud Run
  examples/experiment.json Ejemplo del contrato
  firestore.rules         Bloqueo de acceso directo
  firestore.indexes.json  Sin índices del mapa de resultados
  docs/                   Este manual y notas de verificación
```

Flujo: Android obtiene un Firebase ID token mediante Google. La API verifica el token, valida el JSON y ejecuta NumPy en otro proceso. Firestore guarda el experimento asociado al UID verificado. Android solicita la lista y luego el detalle.

Versiones: React Native 0.85.3, React 19.2.3, Firebase RN 27.0.0, Google Sign-In 16.1.5. Docker: Python 3.12.10, NumPy 2.2.6. NumPy es la única biblioteca del cálculo y muestreo; FastAPI y Firebase solo atienden API, autenticación y almacenamiento.

Los SDK, node_modules y cachés se descargan al instalar. Los archivos privados de tu proyecto Google y de la firma se agregan después.

03 / Instala las herramientas

1. Instala Node.js 24 LTS o una versión compatible, como Node 22.11 o posterior. No uses una versión anterior a la exigida por package.json. Instala JDK 17 y Android Studio.

2. En Android Studio abre Settings > Languages & Frameworks > Android SDK. En SDK Platforms instala Android API 36. En SDK Tools activa Show Package Details e instala Build Tools 36.0.0, Android SDK Platform-Tools, Command-line Tools, NDK (Side by side) 27.1.12297006 y CMake 3.22.1.

3. En Settings > Build, Execution, Deployment > Build Tools > Gradle elige JDK 17 como Gradle JDK. Evita que Android Studio elija automáticamente otro JDK incompatible.

4. Define ANDROID_HOME con la ruta de tu SDK en las variables de entorno de Windows. Valor habitual: C:\Users\TU_USUARIO\AppData\Local\Android\Sdk. Agrega platform-tools a Path. JAVA_HOME debe apuntar al directorio de JDK 17, sin \bin.

```
node --version
npm --version
java -version
$env:ANDROID_HOME
```

5. Para desarrollo usa un emulador Android API 36 con imagen Google Play, o un teléfono con servicios Google y depuración USB. Un emulador sin Google Play no permite comprobar Google Login.

Docker Desktop es opcional para compilar Android. Se necesita para ejecutar el contenedor localmente; el despliegue con Cloud Build puede construirse sin Docker en tu PC.

04 / Extrae e instala las fuentes

1. Extrae el ZIP en una ruta corta, por ejemplo C:\dev\MonteCarlo. Evita carpetas sincronizadas y rutas largas: los archivos nativos de Android pueden superar límites de Windows.

2. Abre PowerShell en la carpeta mobile y ejecuta:

```
cd C:\dev\MonteCarlo\mobile
npm ci
npm run typecheck
npm test
```

3. npm ci usa package-lock.json. Conserva ese archivo para repetir la instalación. No copies node_modules de otro equipo. La descarga inicial puede tardar varios minutos.

4. En Android Studio elige Open y selecciona C:\dev\MonteCarlo\mobile\android. Permite la sincronización y la descarga de Gradle. Android Studio creará local.properties con la ruta local del SDK.

Comprobación sin configurar Google

Puedes compilar las fuentes sin google-services.json. En ese caso aparece una pantalla para conectar el laboratorio. Esto permite comprobar el proyecto nativo, pero todavía no permite iniciar sesión ni guardar experimentos.

No ejecutes de nuevo react-native init: el ZIP ya contiene el proyecto Android, sus fuentes Kotlin y el Gradle Wrapper. No hace falta convertirlo desde una página web.

05 / Configura Google Login

1. Abre console.firebase.google.com y crea o añade Firebase a tu proyecto Google Cloud. Anota el PROJECT_ID exacto; no lo confundas con el nombre visible ni con el número del proyecto.

2. En Project settings > Your apps > Add app > Android registra el paquete **com.montecarlolab**. Usa ese mismo identificador durante todo este manual.

3. Desde mobile obtén SHA-1 y SHA-256 de la clave debug incluida:

```
keytool -list -v -keystore android/app/debug.keystore `
  -alias androiddebugkey -storepass android
```

4. Agrega ambas huellas en la configuración de la app Android en Firebase. En Authentication > Sign-in method habilita Google y selecciona un correo de soporte.

5. Si Google Cloud pide configurar Google Auth Platform/OAuth consent, completa nombre, correo y audiencia. Mientras el proyecto esté en modo Testing, añade tu cuenta y las de tus probadores a Test users.

6. Vuelve a descargar google-services.json después de habilitar Google y registrar las huellas. Guárdalo exactamente aquí:

```
C:\dev\MonteCarlo\mobile\android\app\google-services.json
```

Debe corresponder a com.montecarlolab e incluir un cliente OAuth Web, client_type 3. El script de configuración leerá ese client_id automáticamente. La API usa Firebase ID tokens, no el token de Google directamente.

La clave de release tiene otras huellas. Las registrarás al firmar el APK en el paso 09. Nunca introduzcas una clave privada de servicio en Android.

06 / Crea y protege Firestore

1. En Firebase Console abre Build > Firestore Database > Create database. Crea la base **(default)** en modo Native/Standard. Elige una región cercana, por ejemplo us-central1, y modo producción.

2. El almacenamiento usa montecarlo_users/{uid}, con un mapa experiments. Cada usuario puede tener hasta diez entradas. No tienes que crear colecciones o documentos manualmente.

3. Instala Firebase CLI e inicia sesión. Ejecuta estos comandos desde la raíz MonteCarlo, que contiene firebase.json:

```
npm install --global firebase-tools
firebase login
firebase deploy --only firestore --project TU_PROJECT_ID
```

4. El despliegue aplica firestore.rules y firestore.indexes.json. Las reglas deniegan lecturas y escrituras directas desde clientes. El mapa experiments queda exento de indexación; la API no hace consultas por campos internos.

Cómo accede la API

El SDK Admin de Google en Cloud Run utiliza la identidad del servicio y permisos IAM. No depende de las reglas para clientes. El script de despliegue asigna datastore.user para Firestore y firebaseauth.viewer para comprobar revocación del token.

No cambies las reglas a allow read, write: if true para arreglar un error de permisos. Si el backend no puede leer, revisa su cuenta de servicio y el PROJECT_ID. No compartas las credenciales de esa cuenta con la app.

07 / Despliega el backend

1. En Google Cloud selecciona el mismo proyecto Firebase. Vincula facturación y abre Cloud Shell. Necesitas permisos para habilitar APIs, crear cuentas, asignar IAM, construir imágenes y desplegar Cloud Run (por ejemplo, propietario del proyecto de desarrollo).

2. Sube el ZIP usando el menú de Cloud Shell > Upload. Extrae el archivo y entra en su carpeta raíz. Ejemplo:

```
unzip MonteCarlo_Android_Fuentes.zip
cd MonteCarlo
bash scripts/deploy.sh TU_PROJECT_ID us-central1
```

3. El script habilita APIs, crea el repositorio de imágenes montecarlo y la cuenta montecarlo-api, asigna permisos y construye backend/Dockerfile con Cloud Build. Después despliega el contenedor en Cloud Run.

4. Espera a que finalice. Copia la URL HTTPS que devuelve, por ejemplo https://montecarlo-api-XXXXX.run.app. No agregues /v1/experiments al configurar la app.

```
curl https://TU-SERVICIO.run.app/health
```

Respuesta esperada: {"status":"ok"}. /health solo comprueba el proceso HTTP. Las operaciones de datos requieren token y también comprueban el acceso real a Firestore.

Configuración del servicio: 1 CPU, 512 MiB, 1 proceso HTTP, concurrencia 4, máximo 3 instancias y HTTP 90 s. Cada instancia ejecuta una simulación a la vez. Firestore impone además una sola simulación por cuenta entre todas las instancias y hasta 25 escenarios por día UTC (capítulo 21).

El acceso público a Cloud Run permite llegar a la API; los endpoints de datos siempre exigen Firebase ID token. Una política de organización que prohíba servicios públicos requiere un diseño IAM/gateway adicional antes de usar esta app.

08 / Conecta Android al servicio

1. Comprueba que google-services.json está en mobile/android/app. En PowerShell, dentro de mobile, ejecuta con tu URL real:

```
npm run configure -- https://TU-SERVICIO.run.app
npm run typecheck
npm test
```

2. El comando genera src/config.ts con la URL, el cliente OAuth Web y configured=true. No almacena contraseñas. Si falla porque falta client_type 3, vuelve al paso 05, habilita Google y descarga otra vez el archivo de Firebase.

3. En Android Studio pulsa Sync Project with Gradle Files. El plugin Google Services se aplica al detectar tu archivo. Después vuelve a compilar para que la configuración forme parte del APK.

Prueba durante el desarrollo

Para depurar con Metro abre una terminal en mobile y ejecuta npm start. Con un emulador o dispositivo conectado, usa Run en Android Studio o npm run android en otra terminal. Para un teléfono USB puedes utilizar adb reverse tcp:8081 tcp:8081.

```
npm start
# En otra terminal dentro de mobile:
npm run android
```

Las builds debug necesitan Metro. Para un APK independiente que puedas instalar sin tu computador, usa la variante release firmada del paso siguiente. Release incluye el código JavaScript y no utiliza Metro.

La URL configurada debe ser HTTPS. Para evitar errores de red y TLS, la primera prueba completa se recomienda contra Cloud Run, también cuando usas un emulador.

09 / Genera tu APK firmado

1. Abre mobile/android en Android Studio. Confirma que la sincronización terminó y selecciona Build > Generate Signed App Bundle / APK (el nombre puede variar ligeramente según la versión). Selecciona APK, no Android App Bundle.

2. En Key store path usa Create new. Crea una clave privada fuera del ZIP de fuentes, por ejemplo C:\claves\azar-release.jks. Define contraseñas, alias azar y los datos del certificado. Conserva una copia segura: la necesitarás para futuras actualizaciones.

3. Obtén las huellas de esa clave:

```
keytool -list -v -keystore C:\claves\azar-release.jks `
  -alias azar
```

4. Registra SHA-1 y SHA-256 de release en Firebase. Descarga de nuevo google-services.json, reemplázalo en android/app, ejecuta otra vez npm run configure con tu URL y sincroniza Gradle.

5. Vuelve al asistente de firma. Selecciona la clave, alias y contraseñas. Elige build variant release y una carpeta de destino. Pulsa Create/Finish. Espera a BUILD SUCCESSFUL y usa Locate para encontrar el APK firmado.

6. Instala el APK en el teléfono. Para instalación por USB:

```
adb install -r C:\ruta\app-release.apk
```

Si ya existe una instalación debug con el mismo paquete y distinta firma, desinstálala antes; Android no permite reemplazarla con otra firma. Tu historial en la nube permanece asociado a la cuenta.

El proyecto no firma release con la clave pública de debug. assembleRelease desde terminal produce un APK sin firmar: no lo distribuyas como si ya fuera instalable. Si publicas en Google Play, registra también la firma de Play App Signing.

10 / Comprueba el flujo completo

1. Abre el APK, toca Continuar con Google y elige una cuenta registrada como probadora si tu OAuth sigue en Testing. Al iniciar sesión la app consulta automáticamente tus experimentos.

2. Toca Nuevo experimento. Selecciona Normal, media 10, desviación 2 y Agregar variable. Se llamará X1. Selecciona Uniforme, mínimo 0, máximo 5 y agrega X2.

3. Deja el dominio opcional vacío. Escribe un título y la expresión X1 + 2*X2. Toca Simular y guardar. Obtendrás exactamente 100 valores finales. Su media suele aproximarse a 15, con variabilidad de Monte Carlo.

4. Abre el experimento y cambia el número de particiones de 10 a 20. El gráfico se actualiza inmediatamente con los mismos datos. Las frecuencias relativas suman 100%; no representan densidad de probabilidad.

5. Vuelve a la tabla. Cierra y abre la app; inicia sesión de nuevo si se solicita. El experimento debe mantenerse y mostrar el mismo color. No debe generarse una nueva simulación al consultarlo.

6. Crea diez experimentos. La app deshabilita crear otro y el backend también rechaza un undécimo. Borra uno y confirma que desaparece al actualizar. Entonces podrás guardar otro.

7. Sal e inicia con otra cuenta. No debe ver los experimentos de la primera. La separación la impone el backend usando UID desde el token, no un ID enviado por Android.

8. Prueba X1/0 y un dominio imposible, como Uniforme(0,1) con dominio [2,3]. La API debe informar error y no guardar resultados parciales.

11 / Parámetros y dominios

Continuas: Uniforme: low < high. Normal: mean libre, std > 0. LogNormal: mean y sigma del logaritmo; sigma > 0. Weibull: shape y scale > 0. Gamma: shape y scale > 0. Beta: alpha y beta > 0. Exponencial: rate > 0, media 1/rate. La exponencial se clasifica correctamente como continua.

Discretas: Bernoulli: p en [0,1]. Categórica: escribe valores numéricos únicos separados por comas y, en el mismo orden, sus probabilidades; deben sumar 1. Admite entre 1 y 50 categorías. Poisson cero-inflada: lam > 0, pi en [0,1]. Logarítmica: serie logarítmica, p en (0,1), soporte 1,2,3...

Poisson generalizada: variante explícita

Se implementa Consul-Jain con dispersión no negativa: 0 < lambda ≤ 100 y 0 ≤ theta ≤ 0.9. Su media es lambda/(1-theta) y su varianza lambda/(1-theta)^3. Con theta=0 se reduce a Poisson. No se incluye dispersión negativa en esta versión.

```
P(K=0) = exp(-lambda)
P(K=k) = lambda * (lambda+theta*k)^(k-1)
         * exp(-lambda-theta*k) / k!     para k >= 1
```

El muestreo usa generaciones de descendientes Poisson construidas solo con NumPy: inmigrantes Poisson(lambda), hijos Poisson(theta por generación) y suma de generaciones hasta extinguirse.

Dominio opcional

Mínimo y máximo son inclusivos y deben cumplir mínimo < máximo si están ambos. Se intersectan con el soporte natural. Se aplica rechazo, conservando la distribución condicionada al intervalo; no se recorta ni se reescala. Un dominio de masa nula o demasiado pequeña falla por presupuesto de intentos o por tiempo. Usa punto para decimales.

12 / Escribe expresiones seguras

Los nombres disponibles son X1 a X3. Agrega cada variable antes de usarla. Quitar una variable que aparece en la expresión obliga a corregirla. Los parámetros pueden redefinirse quitando y volviendo a agregar la variable.

```
X1 + 2*X2
sqrt(abs(X1)) + log(X2)
exp(-X1)
(X1 + X2)^2
X1 - X1
```

Operadores: +, -, *, /, ^. Usa * explícitamente: 2X1 es inválido. ^ es potencia y se asocia a la derecha: X1^2^3 = X1^(2^3). La potencia tiene prioridad sobre el signo: -X1^2 = -(X1^2).

Funciones de un argumento: abs, sqrt, log (logaritmo natural), exp, sin y cos. Usa paréntesis; sin y cos reciben radianes. No se aceptan asignaciones, atributos, llamadas arbitrarias ni fragmentos de JavaScript o Python.

Android convierte la expresión a un árbol JSON (AST). El backend vuelve a validar cada nodo y lo evalúa con operaciones NumPy de una lista permitida. No se ejecuta eval y no se necesita SymPy.

Cada realización utiliza una muestra de cada variable independiente. Si X1 se repite, se reutiliza la misma muestra dentro de esa realización; por eso X1-X1 produce cien ceros. No se calcula un producto cartesiano entre muestras.

Se rechaza el experimento completo si cualquier resultado no es finito o supera magnitud 1e100, por ejemplo división por cero, raíz de un negativo, logaritmo no definido o desbordamiento de exp. No se reemplazan silenciosamente por ceros ni se eliminan observaciones.

13 / Formato que recibe la API

POST /v1/experiments recibe JSON con seis campos obligatorios, sin campos desconocidos. La app usa protocolo 2; protocolo 1 sigue disponible para datos anteriores. Usa Authorization: Bearer FIREBASE_ID_TOKEN e Idempotency-Key: UUID. examples/experiment-1d.json, -2d.json y -3d.json contienen solicitudes completas.

```
{
  "version": 2,
  "title": "Vector compartido",
  "samples": 100,
  "variables": [
    {"name":"X1", "distribution":"normal",
     "params":{"mean":10,"std":2}, "domain":{}}
  ],
  "expressions": [
    {"var":"X1"},
    {"op":"mul", "args":[{"const":2},{"var":"X1"}]},
    {"op":"sub", "args":[{"var":"X1"},{"var":"X1"}]}
  ]
}
```

Nodos AST: {"const":2} es una constante; {"var":"X1"} es una referencia; {"op":"add","args":[...]} es una operación. Binarios: add, sub, mul, div, pow. Unarios: neg, abs, sqrt, log, exp, sin, cos.

Las solicitudes usan una clave de idempotencia persistente durante el reintento del formulario. Si el guardado ya ocurrió con esa clave y ese contenido, se devuelve el mismo experimento. Una nueva sesión o formulario puede generar otra clave; ante una interrupción de red consulta primero la tabla para evitar duplicados voluntarios.

La API no acepta resultados calculados por el cliente. Genera 100 vectores en el backend. Cada componente requiere una variable; el total es de 100, 200 o 300 números. El cliente tampoco decide a qué UID se guardan.

14 / Consultas, datos e histograma

```
GET    /health                  Estado HTTP, sin autenticación
GET    /v1/experiments           Metadatos para la tabla
POST   /v1/experiments           Simula y guarda
GET    /v1/experiments/{uuid}    Definición y resultados
DELETE /v1/experiments/{uuid}    Borra de Firestore
GET    /v1/usage                 Cupo diario y estado de cuenta
```

Todos los endpoints de datos exigen Firebase ID token. En cada consulta el UID se obtiene del token verificado. Una cuenta distinta recibe lista propia y 404 al intentar consultar o borrar un experimento ajeno.

Qué se guarda: ID, título, fecha UTC, color, definición de variables, parámetros, dominio, AST de las componentes y cien vectores finales, en una lista plana de 100/200/300 números por filas: x0,y0,z0,x1,y1,z1,... Dimensions indica 1, 2 o 3. No se almacena el vector que tomó cada variable. Esos valores intermedios se descartan al terminar el proceso.

La lista omite resultados y definición. Al tocar Abrir se pide el detalle al servidor y se construye el histograma en Android. El color se elige aleatoriamente una vez y se guarda; pueden coincidir colores por azar.

Frecuencias relativas

En 1D, entre 1 y 50 intervalos de igual ancho. Para 2D/3D consulta el capítulo 20. Cada barra vale conteo del intervalo dividido entre 100; las barras suman 1. El eje y muestra proporción expresada como porcentaje, no conteo total. El último intervalo incluye el máximo; los anteriores son [a,b).

Cambiar particiones redistribuye los mismos cien resultados. Si todos son iguales se amplía el rango del eje para poder dibujar. Debajo del gráfico se muestra la tabla de intervalos y porcentajes, además de media, mínimo y máximo.

15 / Límites y ejecución

Entrada: 1 a 3 variables únicas X1...X3, exactamente 100 realizaciones, protocolo 1 o 2, título de 1 a 80 caracteres, JSON de hasta 32 KiB, números finitos de magnitud hasta 1e9. Booleanos no cuentan como números. Se rechazan claves duplicadas y campos desconocidos.

Expresión: hasta 500 caracteres en Android; en la API hasta 127 nodos y profundidad 20 por componente (1 a 3). Al menos una referencia a una variable aleatoria definida. Solo operadores y funciones autorizados.

Almacenamiento: máximo 10 experimentos por cuenta. La reserva y el guardado comprueban el límite mediante transacciones Firestore. Esto impide que solicitudes simultáneas desde distintas instancias excedan el límite. Borrar también modifica Firestore mediante transacción.

Tiempo: el motor se ejecuta en un proceso hijo con plazo de 60 segundos de reloj, incluyendo su inicialización. Al vencer se termina el proceso, se espera su salida y no se guarda. No es un timeout de hilo que deje el cálculo ejecutándose por detrás.

Autenticación, red y escritura de base de datos añaden tiempo al cálculo. El HTTP de Cloud Run y la espera de Android son de 90 segundos. Si una respuesta se pierde, puede haberse guardado antes: vuelve a consultar la lista antes de crear otro formulario.

Códigos: 401 sesión inválida; 404 experimento ausente; 408 simulación vencida; 409 límite o conflicto de clave; 413 cuerpo grande; 422 formato, parámetros o cálculo inválido; 429 cuenta/instancia ocupada o cuota diaria; 503 servicio temporalmente no disponible.

Las variables base son independientes; las componentes pueden ser dependientes por compartirlas. No hay semilla editable en la app. Las pruebas sí usan semillas para comprobar el motor de manera reproducible.

16 / Ejecuta Python con Docker

Esta alternativa sirve para inspeccionar y probar el backend local. El despliegue de Cloud Run ya utiliza el mismo Dockerfile y no necesita Docker Desktop en tu PC.

1. Instala y abre Docker Desktop. En PowerShell entra en backend y construye:

```
cd C:\dev\MonteCarlo\backend
docker build -t montecarlo-api .
```

2. Para probar cálculos sin credenciales:

```
docker run --rm montecarlo-api python -c `
  "import numpy; print(numpy.__version__)"
docker run --rm `
  -v "${PWD}/tests:/srv/tests:ro" montecarlo-api `
  python -m unittest discover -s tests -p test_engine.py -v
```

3. Para usar endpoints con Firestore, instala Google Cloud CLI, inicia credenciales locales y apunta Docker al archivo ADC, sin copiarlo a la imagen:

```
gcloud auth application-default login
$env:GOOGLE_CLOUD_PROJECT="TU_PROJECT_ID"
$env:ADC_FILE="$env:APPDATA/gcloud/application_default_credentials.json"
docker compose up --build
```

4. Prueba http://localhost:8080/health. Los endpoints de datos siguen requiriendo un Firebase ID token real del mismo proyecto, de una sesión con proveedor Google. Las credenciales ADC son para el backend, no sustituyen el token del usuario.

La app de release exige HTTPS. Para la comprobación móvil usa Cloud Run. El contenedor fija Python 3.12.10, NumPy 2.2.6, usa un usuario sin privilegios y un solo hilo BLAS. No contiene SciPy ni SymPy.

17 / Pruebas y alcance verificado

Las comprobaciones automáticas vienen con las fuentes. Se pueden repetir antes de modificar o publicar. No necesitan una cuenta Google para probar cálculo, parser, histograma y manejo de la API con almacenamiento simulado.

```
# Desde mobile
npm run typecheck
npm test

# Desde backend, con Python 3.12 instalado
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements-dev.txt
.venv/Scripts/python.exe -m unittest discover -s tests -v
```

Motor: prueba las 12 distribuciones, 100 resultados finitos, medias teóricas y varianza de Poisson generalizada; comprueba reutilización de una variable, combinaciones, rechazo de dominios imposibles, validación y errores numéricos.

API: comprueba autenticación requerida, listas y detalle, aislamiento de cuentas, cuota, borrado, idempotencia, JSON inválido, estado ocupado y ausencia de guardado si se agota el tiempo. Sus pruebas usan un almacén en memoria, por lo que no certifican un proyecto Firestore real.

Proceso: se provoca un cálculo que duerme más que el plazo de prueba, se comprueba que es terminado y que no quedan procesos hijos. El plazo de producción sigue siendo 60 segundos.

Android: prueba precedencia del parser, rechazo de expresiones arbitrarias, parámetros iniciales de las 12 distribuciones y suma de frecuencias relativas, incluidos datos constantes.

Consulta docs/VERIFICACION.md para el resultado exacto de los chequeos realizados en esta entrega. La prueba real de Google Login, IAM, Cloud Run y Firestore debe hacerse con tu proyecto siguiendo el paso 10.

18 / Soluciona errores frecuentes

Google Login falla o devuelve DEVELOPER_ERROR

Comprueba paquete com.montecarlolab, cliente OAuth Web tipo 3, proveedor Google activado y SHA-1 de la firma usada por ese APK. Debug, release y Play App Signing tienen certificados distintos. Descarga el JSON de nuevo, ejecuta configure y recompila. Usa un dispositivo con Google Play Services y una cuenta permitida por OAuth.

SDK, Java, Gradle o rutas

Selecciona JDK 17 en Android Studio; instala SDK 36, Build Tools 36.0.0, NDK 27.1.12297006 y CMake 3.22.1. Abre mobile/android, no solo la raíz. Si falla CMake por rutas largas, extrae en C:\dev\MonteCarlo. La primera sincronización requiere Internet y acceso a Maven y Gradle.

API 401, 403 de plataforma o 503

401: sesión Firebase válida del mismo proyecto y proveedor Google. Sal e inicia sesión otra vez. Un 403 de Cloud Run puede indicar que falta invocación pública o una política organizativa. 503: revisa logs de Cloud Run, Firestore (default), PROJECT_ID y roles datastore.user y firebaseauth.viewer de la cuenta del servicio.

Falló la construcción o el despliegue de Cloud Build

Verifica facturación, APIs y permisos de tu cuenta para asignar IAM. El script consulta la cuenta predeterminada de Cloud Build y le asigna cloudbuild.builds.builder. Espera propagación de IAM y reintenta. Las organizaciones pueden requerir una cuenta de compilación aprobada por su administrador.

No hay datos o el histograma parece distinto

Usa Actualizar tabla y la misma cuenta Google. Cambiar particiones modifica las barras, no los cien resultados. Con solo cien observaciones habrá variación visible. Si se perdió la respuesta al guardar, consulta la tabla antes de iniciar otro experimento.

19 / Referencias oficiales

Fuentes consultadas para configuración e integración. Los nombres de menús de las consolas pueden variar. Las versiones concretas del proyecto están en package.json, package-lock.json, Dockerfile y requirements.txt.

React Native y Android

reactnative.dev/docs/set-up-your-environment
Entorno nativo Android, Java, SDK y herramientas.

reactnative.dev/docs/signed-apk-android
Firma y distribución Android.

Google y Firebase

rnfirebase.io/auth/social-auth
Integración nativa de Google con Firebase Authentication.

firebase.google.com/docs/auth/admin/verify-id-tokens
Verificación de Firebase ID tokens en el servidor.

firebase.google.com/docs/firestore/manage-data/transactions
Transacciones y control de concurrencia.

cloud.google.com/run/docs/deploying
Despliegue de imágenes en Cloud Run.

NumPy

numpy.org/doc/2.2/reference/random/generator.html
Generador aleatorio y distribuciones. Poisson generalizada se construye en el motor mediante ramificación; no se invoca una biblioteca estadística adicional.

Conserva tus claves de firma fuera del proyecto compartido. El ZIP contiene las fuentes y las instrucciones; el historial de experimentos pertenece a tu base Firestore y requiere tu cuenta para consultarlo.

20 / Vectores y gráficos nativos

1. Crea las variables base (máximo tres entre todas las componentes). En Nuevo experimento elige 1D, 2D o 3D. Escribe una expresión para X, otra para Y si corresponde y otra para Z en 3D. Los ejes X/Y/Z son componentes de salida; X1...X3 son las variables base.

```
1D: X = X1 + X2              100 números
2D: X = X1, Y = 2*X1         200 números
3D: X = X1, Y = X2, Z = X1*X2
                             300 números
```

2. Toca Simular y guardar. Se muestrean las variables una sola vez por cada una de las cien realizaciones. Esos mismos valores se usan en todas las componentes. En (X1,2*X1), Y siempre es el doble de X; no se vuelven a sortear los valores para Y.

3. Abre el experimento. En 2D, la base representa intervalos de X e Y y la altura de cada barra es conteo/100. En 3D, X/Y/Z son intervalos de las tres componentes: cada voxel ocupado tiene intensidad proporcional a su frecuencia relativa. No se usa la frecuencia como una cuarta coordenada espacial.

4. Configura particiones por eje: 1D de 1 a 50; 2D de 1 a 12; 3D de 1 a 8. Usa Girar para cambiar el ángulo. En 3D selecciona Todo o un corte Z. Los cortes mantienen conteo/100, sin renormalizar; su suma puede ser menor que 100%.

La tabla muestra intervalos y frecuencias de las celdas ocupadas; las vacías valen cero. Todas las celdas del histograma completo suman 100%. Estos ajustes son locales y no gastan escenarios ni escriben en Firestore.

Visualización: Views de React Native para 1D y una View Android propia en Kotlin con android.graphics.Canvas para 2D/3D. Sin WebView, Three.js ni navegador. No se recibió el documento visual mencionado; las figuras son una interpretación funcional, no una reproducción de ese material.

21 / Bloqueo y 25 escenarios diarios

Una cuenta, un cálculo activo: antes de iniciar NumPy, el servidor reserva el UID y aumenta su contador en una sola transacción de Firestore. Si ya hay reserva, devuelve 429. Esto funciona aunque la petición venga de otro teléfono, se modifique la app o Cloud Run la atienda en otra instancia.

El botón deshabilitado solo ayuda a usar la app. La protección efectiva está en el servidor y en las reglas que niegan escrituras directas a Firestore. Durante el cálculo también se bloquea borrar experimentos; consultar lista, detalle y uso sigue permitido.

25 escenarios por día UTC: un escenario de 1, 2 o 3 componentes consume un cupo. Se cuenta cada intento válido admitido, aunque falle el cálculo o alcance 60 segundos. Borrar no devuelve cupo. Las solicitudes rechazadas antes de admitirlas no consumen: sesión, formato, reserva activa, capacidad o cuota.

Un reintento de una clave ya guardada con idéntico contenido devuelve ese experimento sin calcular ni aumentar el contador. Una clave con otro contenido da 409. Si no se guardó y se vuelve a admitir, es un nuevo intento y consume cupo.

```
GET /v1/usage   # Requiere Firebase ID token
{
  "day_utc": "2026-10-20", "used": 7, "limit": 25,
  "remaining": 18, "running": false,
  "resets_at": "2026-10-21T00:00:00+00:00"
}
```

El reinicio es a las 00:00 UTC según el servidor; cambiar el reloj o zona del teléfono no lo altera. La app consulta el cupo al iniciar sesión y al actualizar la tabla. Si otra sesión termina un cálculo, toca Actualizar tabla.

Recuperación: la reserva vence a los 120 segundos si cae el proceso. El cálculo dura como máximo 60; se reduce su presupuesto si la admisión se demoró. Un token de propietario impide que una ejecución antigua guarde o libere la reserva nueva. Guardado y liberación son atómicos. Si falla la limpieza por red, espera hasta vencer la reserva.

22 / Comprueba la ampliación

1. Dependencia: agrega X1 Normal(0,1), selecciona 2D y escribe X=X1, Y=2*X1. Guarda y abre. Las celdas ocupadas deben seguir una diagonal. Hay cien pares (200 números), no doscientos sorteos independientes.

2. Tres componentes: agrega X2 Uniforme(0,1), elige 3D y escribe X=X1, Y=X2, Z=X1*X2. Guarda: cien ternas y 300 números. Cambia particiones, gira y revisa cada corte de Z contra la tabla. Todo debe sumar 100%; un corte aislado puede sumar menos.

3. Bloqueo: usa la misma cuenta en dos dispositivos o dos clientes autenticados. Mientras uno simula, intenta iniciar otra simulación y borrar desde el otro. El servidor debe responder 429. No basta con comprobar que el botón esté deshabilitado.

4. Cupo: en una cuenta de prueba crea y borra escenarios para poder llegar a 25 intentos. La petición 26 debe responder 429 aunque la tabla esté vacía. Verifica used=25 y remaining=0. Un intento admitido que falle también cuenta; una petición con formato inválido no.

5. Reintentos: con un cliente de pruebas, repite la misma petición y la misma Idempotency-Key después de guardar. Debe devolver el mismo ID, color y resultados sin aumentar used. Cambia el contenido con esa clave: debe dar 409.

6. Compatibilidad: experimentos existentes con version=1 y expression se abren como 1D. Las nuevas solicitudes de la app usan version=2 y expressions. Actualiza primero el backend y luego el APK.

Pruebas incluidas

Las pruebas de Store ejecutan su lógica real con transporte simulado: cuotas, cambio de fecha UTC, reservas, propietario antiguo y admisiones concurrentes. tests/test_firestore.py comprueba admisiones entre clientes del SDK y cuota diaria con un emulador local; se omite si no está configurado. El detalle de lo ejecutado está en docs/VERIFICACION.md.

No introduzcas resultados de prueba en una cuenta de uso normal si necesitas conservar su cupo diario. No se proporciona un endpoint de cliente para modificar contadores o saltar el límite.